

SBA301

SLOT 04 - PRACTICE PROJECT GUIDE

Interactive Orchid Explorer

ReactJS + Props + State + Hooks + Context + React-Bootstrap + IntelliJ IDEA

Learning transition: Slot 03: structure the UI -> Slot 04: make data flow, change, and drive the UI. This project consolidates Props, State/useState, event-driven UI, Context/useContext, Hook rules, debugging, verification, and Git evidence.

Course	SBA301 - Integrate Single Page Application with Spring Boot
Baseline	SBA301-40 v2
Project level	Intermediate / consolidation project
IDE	IntelliJ IDEA
Recommended starting point	Verified Slot 03 Orchid Gallery project
Core scope	Props, State, useState, events, Context/useContext; useEffect preview only

1. Project overview and learning purpose

Interactive Orchid Explorer is a medium-difficulty ReactJS project that upgrades the static component-based Orchid Gallery from Slot 03 into a reusable, interactive application. The project deliberately keeps data local to the React client: no Spring Boot, REST API, database, Router, Redux, or external state library is introduced.

The central questions are: Where does the data originate? Who owns it? Who needs it? Who may change it? Students must be able to answer those questions before they consider the project complete.

Item	Project decision
Domain	Orchid discovery / learning gallery
Main user	Student / orchid enthusiast
Difficulty	Intermediate
Data source	Static JavaScript array in src/data/orchids.js
Reusable unit	OrchidCard receives orchid data through Props
Local state	Detail modal, favorite toggle, search text, special-only filter
Shared context	Current user information shared to distant UI components
Verification	Browser UI + DevTools Console + state/data-flow trace + Git checkpoint

2. Learning objectives

- Create one reusable component that receives object Props and renders different data.
- Explain Props as parent-to-child, read-only component input.
- Use useState to store local changing data and connect it to user events.
- Trace event -> handler -> state updater -> new state -> render -> updated UI.
- Distinguish Props, local State, derived data, and Context by ownership and scope.
- Use createContext, Provider, and useContext for a deliberately shared value.
- Apply the Rules of Hooks and identify invalid Hook placement.
- Reuse React-Bootstrap components from Slot 03 without letting styling hide data-flow problems.
- Debug with terminal evidence, browser console evidence, and one-change-at-a-time verification.
- Create a meaningful Git checkpoint after the dynamic application works.

Core rule: A visually attractive UI is not enough. The student must explain the data flow and state ownership without AI.

3. Functional requirements

ID	Function	Slot 04 concept
F01	Display a responsive list of orchids from one data array	Props + composition
F02	Use the same OrchidCard component for all orchids	Reusable Props
F03	Open/close an orchid detail Modal	useState + onClick
F04	Toggle Favorite for each OrchidCard	local State
F05	Search orchids by name/category	useState + derived filtered data
F06	Filter to show only special orchids	boolean State + derived data
F07	Show the current visible-result count	derived data
F08	Share current-user information in Navbar and Footer	Context + useContext
F09	Show an informative empty-state Alert when no orchid matches	state-driven conditional UI
F10	Provide repeatable run, test, debug, build, and Git evidence	verification discipline

4. Technologies and tools

Technology / tool	Purpose	Scope
ReactJS	Build component-based UI	Core
Vite	Development server/build tooling	Existing environment
JavaScript / JSX	Application logic and UI description	Core
React-Bootstrap	Container/Row/Col/Card/Button/Modal/Form/Alert	Core UI vehicle

Technology / tool	Purpose	Scope
IntelliJ IDEA	Edit, terminal, run configuration, inspection	Required IDE for this guide
Node.js + npm	React tooling and package execution	Environment prerequisite
Git	Version-control checkpoint	Required evidence
Browser DevTools	Console and UI verification	Required evidence
useEffect	Optional document-title micro-challenge	Preview only

5. Architecture before coding

Target component tree

```

App
|-- useContext.Provider
|   |-- AppNavbar      <-- useContext(currentUser)
|   |-- HeroSection
|   |-- OrchidExplorer
|       |-- Search / special filter state
|       |-- OrchidCard  <-- orchid prop
|           |-- showDetail state
|           |-- isFavorite state
|       `-- OrchidCard ...
|-- AppFooter      <-- useContext(currentUser)

```

Core Props + State flow

```

DATA ARRAY
|
| Props
v
OrchidExplorer
|
| orchid object
v
OrchidCard
|
| Local State
+--> showDetail
`--> isFavorite

```

Context flow

```

CURRENT USER
|
v
UserContext.Provider
|-----|
v       v
AppNavbar AppFooter
useContext useContext

```

Design checkpoint: Before coding, the student must be able to explain why orchid data is passed by Props, why Modal/Favorite are local State, and why the current user is chosen as the Context demo.

6. Target folder structure

```

interactive-orchid-explorer/
|-- public/
|   |-- images/
|       |-- orchid-01.jpg
|       |-- orchid-02.jpg
|       |-- ...
|-- src/
|   |-- components/
|       |-- AppNavbar.jsx

```

```

| | | -- HeroSection.jsx
| | | -- OrchidExplorer.jsx
| | | -- OrchidCard.jsx
| | | `-- AppFooter.jsx
| | -- context/
| | | `-- UserContext.js
| | -- data/
| | | `-- orchids.js
| | -- App.jsx
| | -- main.jsx
| | `-- index.css
|-- package.json
|-- vite.config.*
`-- .gitignore

```

7. Summary of implementation steps

1. Prepare / open the Slot 03 project in IntelliJ IDEA.
2. Verify Node, npm, Git, package.json, and the baseline application.
3. Verify or install React-Bootstrap and Bootstrap.
4. Create the target folders and inspect the intended component tree.
5. Create the orchid data module.
6. Create UserContext.
7. Wrap the app with UserContext.Provider.
8. Create AppNavbar and read the shared user with useContext.
9. Create HeroSection.
10. Create OrchidCard with object Props.
11. Add local showDetail State and Modal interaction.
12. Add local isFavorite State and toggle interaction.
13. Create OrchidExplorer with searchText State.
14. Add specialOnly boolean State.
15. Compute visibleOrchids as derived data.
16. Render the list with one reusable OrchidCard.
17. Add result count and empty-state Alert.
18. Create AppFooter and read UserContext.
19. Compose all components in App.jsx.
20. Add minimal CSS and verify responsive layout.
21. Run systematic manual test cases.
22. Use Browser Console + React data-flow debugging checklist.
23. Create IntelliJ npm Run Configuration.
24. Run npm build and preview verification.
25. Create Git checkpoint and evidence package.
26. Complete the Human Verification Gate.
27. Optional: useEffect document-title challenge.

8. Detailed implementation guide

Step 1 - Open and verify the project in IntelliJ IDEA

1. Open IntelliJ IDEA.
2. Choose File -> Open and select the existing Slot 03 project folder. If you intentionally create a fresh project, use the course-approved Vite workflow before continuing.
3. Wait until IntelliJ finishes indexing.
4. Open View -> Tool Windows -> Terminal.
5. Confirm the terminal working directory is the project root containing package.json.

Run in the IntelliJ Terminal

```
node -v
npm -v
git --version
```

Windows quick check (PowerShell/CMD may vary)

```
dir package.json
```

Checkpoint	Expected result	Student evidence
IDE + tooling readiness	IntelliJ opens the project; node/npm/git commands work; package.json exists in current folder.	PASS / FAIL + note

Stop rule: If package.json is not in the current directory, do not run npm install or npm run dev yet. Correct the terminal directory first.

Step 2 - Baseline run before changing code

```
npm install
npm run dev
```

1. Wait until the development server prints a local URL.
2. Open that URL in the browser.
3. Confirm the Slot 03 Orchid Gallery still loads.
4. Press F12 -> Console and confirm there is no relevant application error.
5. Record the baseline as PASS before editing.

Checkpoint	Expected result	Student evidence
Baseline	Existing app loads successfully and Console has no relevant error.	URL + screenshot / note

Step 3 - Verify React-Bootstrap dependency and CSS

Slot 04 reuses React-Bootstrap from Slot 03. If the packages are already present, do not reinstall merely for repetition. Verify package.json first. If missing, install them.

```
npm install react-bootstrap bootstrap
```

Ensure this import exists in main.jsx or the course-approved entry file

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Checkpoint	Expected result	Student evidence
React-Bootstrap	Application still runs and Bootstrap-styled components display correctly.	PASS / FAIL + note

Step 4 - Create the folder skeleton

1. Under src, create components if it does not already exist.
2. Create src/data.
3. Create src/context.
4. Keep App.jsx and main.jsx at src root unless the course project already uses another approved structure.

Checkpoint	Expected result	Student evidence
Folder structure	src/components, src/data, and src/context exist and are visible in IntelliJ Project view.	PASS / FAIL + note

Step 5 - Create the orchid data module

Create src/data/orchids.js. Use course/Lab orchid data if provided. The following data is a self-contained practice dataset; replace image paths with files that actually exist in public/images.

```
export const orchids = [
  {
    id: 1,
    orchidName: "Taichung Beauty",
    category: "Cattleya",
    origin: "Taiwan",
    color: "Purple",
    rating: 4.8,
    isSpecial: true,
    image: "/images/orchid-01.jpg",
    description: "A colorful orchid used for Slot 04 practice."
  },
  {
    id: 2,
    orchidName: "Golden Sunrise",
    category: "Dendrobium",
    origin: "Thailand",
    color: "Yellow",
    rating: 4.5,
    isSpecial: false,
    image: "/images/orchid-02.jpg",
    description: "A bright Dendrobium sample for reusable Card rendering."
  },
  {
    id: 3,
    orchidName: "White Moon",
    category: "Phalaenopsis",
    origin: "Vietnam",
    color: "White",
    rating: 4.9,
    isSpecial: true,
    image: "/images/orchid-03.jpg",
    description: "A Phalaenopsis sample for Props and filtering practice."
  },
  {
    id: 4,
    orchidName: "Pink Dream",
    category: "Phalaenopsis",
    origin: "Vietnam",
    color: "Pink",
    rating: 4.4,
    isSpecial: false,
    image: "/images/orchid-04.jpg",
    description: "A second Phalaenopsis sample to demonstrate repeated UI."
  },
  {
    id: 5,
    orchidName: "Emerald Star",
    category: "Cymbidium",
    origin: "Japan",
    color: "Green",
    rating: 4.7,
    isSpecial: true,
    image: "/images/orchid-05.jpg",
  }
]
```

```

    description: "A special Cymbidium practice entry."
  },
  {
    id: 6,
    orchidName: "Orange Flame",
    category: "Dendrobium",
    origin: "Singapore",
    color: "Orange",
    rating: 4.3,
    isSpecial: false,
    image: "/images/orchid-06.jpg",
    description: "A Dendrobium sample for search and filter testing."
  }
];

```

1. Put matching image files under public/images, or update image to real existing paths.
2. Save orchids.js.
3. Do not put the array into State yet; it is initial static source data for this exercise.

Checkpoint	Expected result	Student evidence
Data module	orchids.js has six unique IDs and all referenced image paths are valid or intentionally replaced.	PASS / FAIL + note

Step 6 - Create useContext

Create src/context/UserContext.js. Context is used only for a small shared current-user value so that both Navbar and Footer can read it without a prop chain.

```

import { createContext } from "react";

const UserContext = createContext(null);

export default UserContext;

```

Checkpoint	Expected result	Student evidence
Context object	UserContext exports successfully; no Hook is used here.	PASS / FAIL + note

Step 7 - Provide the shared user in App.jsx

Create a simple currentUser object in App. App owns the value for this demo and provides it to descendants.

```

import UserContext from "../context/UserContext";

const currentUser = {
  name: "SBA301 Student",
  role: "Learner"
};

function App() {
  return (
    <UserContext.Provider value={currentUser}>
      {/* child components will be composed here */}
    </UserContext.Provider>
  );
}

export default App;

```

Checkpoint	Expected result	Student evidence
Provider	Application compiles. A Provider surrounds the descendants that will consume currentUser.	PASS / FAIL + note

Concept check: Provider makes a value available to descendants; it does not persist the user to a database.

Step 8 - Create AppNavbar and consume Context

```
import { useContext } from "react";
import { Container, Navbar } from "react-bootstrap";
import UserContext from "../context/UserContext";

function AppNavbar() {
  const currentUser = useContext(UserContext);

  return (
    <Navbar bg="dark" data-bs-theme="dark">
      <Container>
        <Navbar.Brand>Interactive Orchid Explorer</Navbar.Brand>
        <Navbar.Text>
          {currentUser ? `Hello, ${currentUser.name}` : "Guest"}
        </Navbar.Text>
      </Container>
    </Navbar>
  );
}

export default AppNavbar;
```

Verify that useContext is called at the top level of the React function component, not inside an if, loop, or click handler.

Checkpoint	Expected result	Student evidence
Context consumer #1	Navbar renders and shows the current user name.	PASS / FAIL + note

Step 9 - Create HeroSection

```
import { Container } from "react-bootstrap";

function HeroSection() {
  return (
    <section className="py-4 bg-light border-bottom">
      <Container>
        <h1 className="mb-2">Explore Orchids</h1>
        <p className="mb-0">
          Props configure reusable cards. State makes the UI interactive.
          Context shares selected app-level information.
        </p>
      </Container>
    </section>
  );
}

export default HeroSection;
```

Checkpoint	Expected result	Student evidence
Static component	HeroSection compiles and has one clear UI responsibility.	PASS / FAIL + note

Step 10 - Create the reusable OrchidCard with Props

Create src/components/OrchidCard.jsx. Start with Props only. Do not add State until the Props baseline works.

```
import { Badge, Button, Card } from "react-bootstrap";

function OrchidCard({ orchid }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Img
        variant="top"
        src={orchid.image}
        alt={orchid.orchidName}
        className="orchid-image"
      />
      <Card.Body className="d-flex flex-column">
```



```

<div className="d-flex justify-content-between align-items-start gap-2">
  <Card.Title>{orchid.orchidName}</Card.Title>
  {orchid.isSpecial && <Badge bg="warning" text="dark">Special</Badge>}
</div>

<Card.Text>
  Category: {orchid.category}<br />
  Origin: {orchid.origin}<br />
  Rating: {orchid.rating}
</Card.Text>

<Button className="mt-auto" variant="primary">
  View Detail
</Button>
</Card.Body>
</Card>
);
}

export default OrchidCard;

```

Checkpoint	Expected result	Student evidence
Props baseline	OrchidCard compiles when it receives a valid orchid object. No prop is modified inside the child.	PASS / FAIL + note

Step 11 - Add detail Modal State to OrchidCard

Now add local State because each card owns its own open/closed detail interaction in this design.

```

import { useState } from "react";
import { Badge, Button, Card, Modal } from "react-bootstrap";

function OrchidCard({ orchid }) {
  const [showDetail, setShowDetail] = useState(false);

  return (
    <>
      <Card className="h-100 shadow-sm">
        <Card.Img
          variant="top"
          src={orchid.image}
          alt={orchid.orchidName}
          className="orchid-image"
        />

        <Card.Body className="d-flex flex-column">
          <div className="d-flex justify-content-between align-items-start gap-2">
            <Card.Title>{orchid.orchidName}</Card.Title>
            {orchid.isSpecial && <Badge bg="warning" text="dark">Special</Badge>}
          </div>

          <Card.Text>
            Category: {orchid.category}<br />
            Origin: {orchid.origin}<br />
            Rating: {orchid.rating}
          </Card.Text>

          <Button
            className="mt-auto"
            variant="primary"
            onClick={() => setShowDetail(true)}
          >
            View Detail
          </Button>
        </Card.Body>
      </Card>

      <Modal show={showDetail} onHide={() => setShowDetail(false)} centered>
        <Modal.Header closeButton>

```

```

    <Modal.Title>{orchid.orchidName}</Modal.Title>
  </Modal.Header>
  <Modal.Body>
    <p><strong>Category:</strong> {orchid.category}</p>
    <p><strong>Origin:</strong> {orchid.origin}</p>
    <p><strong>Color:</strong> {orchid.color}</p>
    <p><strong>Rating:</strong> {orchid.rating}</p>
    <p className="mb-0">{orchid.description}</p>
  </Modal.Body>
  <Modal.Footer>
    <Button variant="secondary" onClick={() => setShowDetail(false)}>
      Close
    </Button>
  </Modal.Footer>
</Modal>
</>
);
}

export default OrchidCard;

```

Mandatory verbal trace: View Detail click -> onClick handler -> setShowDetail(true) -> state becomes true -> OrchidCard renders again -> Modal receives show=true -> Modal appears.

Checkpoint	Expected result	Student evidence
State-driven Modal	View Detail opens only the clicked card detail; Close and X close it; no Console error.	PASS / FAIL + note

Step 12 - Add Favorite local State

Add a second local state variable in OrchidCard. This state belongs to the card for this learning project; it is not persisted.

Add below showDetail state

```
const [isFavorite, setIsFavorite] = useState(false);
```

Add near the View Detail button

```

<Button
  variant={isFavorite ? "danger" : "outline-danger"}
  onClick={() => setIsFavorite(!isFavorite)}
>
  {isFavorite ? "Favorite" : "Add Favorite"}
</Button>

```

Important: Refreshing the browser may reset this Favorite state. useState is client component state, not database persistence.

Checkpoint	Expected result	Student evidence
Favorite state	Button toggles label/variant and affects only the card that owns the state.	PASS / FAIL + note

Step 13 - Create OrchidExplorer with search State

Create src/components/OrchidExplorer.jsx. This component owns search/filter State because those values control which orchids in the collection are shown.

```

import { useState } from "react";
import { Alert, Col, Container, Form, Row } from "react-bootstrap";
import { orchids } from "../data/orchids";
import OrchidCard from "../OrchidCard";

function OrchidExplorer() {
  const [searchText, setSearchText] = useState("");
  const [specialOnly, setSpecialOnly] = useState(false);

  const normalizedText = searchText.trim().toLowerCase();

```

```

const visibleOrchids = orchids.filter((orchid) => {
  const matchesText =
    orchid.orchidName.toLowerCase().includes(normalizedText) ||
    orchid.category.toLowerCase().includes(normalizedText);

  const matchesSpecial = !specialOnly || orchid.isSpecial;

  return matchesText && matchesSpecial;
});

return (
  <Container className="py-4">
    <Row className="g-3 align-items-end mb-4">
      <Col md={8}>
        <Form.Label htmlFor="orchid-search">Search</Form.Label>
        <Form.Control
          id="orchid-search"
          type="text"
          placeholder="Search by name or category..."
          value={searchText}
          onChange={(event) => setSearchText(event.target.value)}
        />
      </Col>

      <Col md={4}>
        <Form.Check
          type="switch"
          id="special-only"
          label="Show special orchids only"
          checked={specialOnly}
          onChange={(event) => setSpecialOnly(event.target.checked)}
        />
      </Col>
    </Row>

    <p className="text-muted">
      Showing {visibleOrchids.length} of {orchids.length} orchids
    </p>

    {visibleOrchids.length === 0 ? (
      <Alert variant="info">No orchid matches the current filters.</Alert>
    ) : (
      <Row className="g-4">
        {visibleOrchids.map((orchid) => (
          <Col key={orchid.id} sm={12} md={6} lg={4}>
            <OrchidCard orchid={orchid} />
          </Col>
        ))}
      </Row>
    )}
  </Container>
);
}

export default OrchidExplorer;

```

Checkpoint	Expected result	Student evidence
Search State	Typing changes searchText and the visible list updates without manual DOM manipulation.	PASS / FAIL + note

Step 14 - Understand derived data: visibleOrchids

visibleOrchids is computed from orchids + searchText + specialOnly. It is not stored as a second State variable because it can be derived from existing values.

Reasoning habit: Before creating State, ask: Can this value be calculated from existing props/state? If yes, it may be derived data rather than additional stored state.

Checkpoint	Expected result	Student evidence
------------	-----------------	------------------

Checkpoint	Expected result	Student evidence
Derived data	Changing either filter recomputes visibleOrchids and the count stays consistent with the displayed cards.	PASS / FAIL + note

Step 15 - Test the special-only boolean State

1. Set search field to empty.
2. Turn Show special orchids only ON.
3. Verify every visible card has isSpecial=true in the source data.
4. Turn it OFF and verify all records can appear again.

Checkpoint	Expected result	Student evidence
Boolean state	Switch ON/OFF changes the result set predictably.	PASS / FAIL + note

Step 16 - Verify one reusable component renders all records

Inspect OrchidExplorer: there is one OrchidCard implementation and map creates repeated component instances with different orchid Props.

```
{visibleOrchids.map((orchid) => (
  <Col key={orchid.id} sm={12} md={6} lg={4}>
    <OrchidCard orchid={orchid} />
  </Col>
))}
```

Checkpoint	Expected result	Student evidence
Reuse	Six data objects can produce six cards without six duplicated OrchidCard component files.	PASS / FAIL + note

Step 17 - Create AppFooter and consume Context again

```
import { useContext } from "react";
import { Container } from "react-bootstrap";
import UserContext from "../context/UserContext";

function AppFooter() {
  const currentUser = useContext(UserContext);

  return (
    <footer className="border-top py-3 mt-4 bg-light">
      <Container className="small text-muted">
        SBA301 Slot 04 practice - current learner: {currentUser?.name ?? "Guest"}
      </Container>
    </footer>
  );
}

export default AppFooter;
```

Checkpoint	Expected result	Student evidence
Context consumer #2	Footer shows the same current user without receiving a currentUser prop.	PASS / FAIL + note

Step 18 - Compose the complete App.jsx

```
import AppFooter from "../components/AppFooter";
import AppNavbar from "../components/AppNavbar";
import HeroSection from "../components/HeroSection";
import OrchidExplorer from "../components/OrchidExplorer";
import UserContext from "../context/UserContext";

const currentUser = {
  name: "SBA301 Student",
  role: "Learner"
}
```

```

};

function App() {
  return (
    <UserContext.Provider value={currentUser}>
      <AppNavbar />
      <main>
        <HeroSection />
        <OrchidExplorer />
      </main>
      <AppFooter />
    </UserContext.Provider>
  );
}

export default App;

```

Checkpoint	Expected result	Student evidence
Composition	App is short, explains the page structure, and does not contain the implementation details of every child component.	PASS / FAIL + note

Step 19 - Verify main.jsx and Bootstrap CSS import

```

import React from "react";
import ReactDOM from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "./index.css";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

If the generated Vite template differs, preserve the actual valid project entry structure. The required concept is that App is rendered at the React root and Bootstrap CSS is imported once in an approved entry point.

Checkpoint	Expected result	Student evidence
Entry point	Application loads after refresh and React-Bootstrap styling remains active.	PASS / FAIL + note

Step 20 - Add minimal CSS

src/index.css

```

body {
  margin: 0;
  min-width: 320px;
  background: #ffffff;
}

.orchid-image {
  width: 100%;
  height: 220px;
  object-fit: cover;
}

.card-title {
  line-height: 1.2;
}

```

Checkpoint	Expected result	Student evidence
Visual baseline	Cards have consistent image height and remain readable at desktop/tablet widths.	PASS / FAIL + note

Step 21 - Run the application and perform first full verification

```
npm run dev
```

Check	Expected	Result
Initial load	Navbar, Hero, filters, six cards, Footer visible	PASS / FAIL
Console	No relevant application error	PASS / FAIL
Search "Dendrobium"	Only Dendrobium cards remain	PASS / FAIL
Search impossible word	Empty-state Alert appears	PASS / FAIL
Special-only ON	Only isSpecial=true cards remain	PASS / FAIL
View Detail	Correct orchid Modal opens	PASS / FAIL
Close Modal	Modal closes	PASS / FAIL
Favorite	Only clicked Card toggles local favorite state	PASS / FAIL
Context	Navbar and Footer show same current user	PASS / FAIL
Resize	Grid remains usable	PASS / FAIL

Step 22 - Debug with evidence, not guesses

Dynamic UI problem

```

  |
  v
Is source data correct?
  |
  v
Is the prop passed with the correct name?
  |
  v
Does the child receive/render it?
  |
  v
Is State initialized correctly?
  |
  v
Does the event fire?
  |
  v
Does the updater execute?
  |
  v
Does rendered JSX depend on the updated State?
  |
  v
If Context is involved: Provider present and correct Context consumed?

```

Symptom	Likely cause	Smallest diagnostic	Verification
orchid is undefined	Wrong prop name / missing prop	Inspect <OrchidCard orchid={...}> and function parameter	Correct values render
Modal never opens	onClick/updater not connected	Temporarily inspect handler path and state variable	showDetail changes false->true
Modal opens during render repeatedly	Setter called during render	Search for setShowDetail(...) outside handler	No render loop
Favorite affects unexpected card	State lifted/shared incorrectly or bad key logic	Check where isFavorite is declared	Each card independent
Search does nothing	onChange or filter expression wrong	Inspect searchText and normalizedText	Visible list changes
Context is null	Consumer outside Provider / wrong import	Trace Provider->consumer tree	Navbar/Footer receive currentUser
Bootstrap looks plain	CSS import missing	Check main.jsx CSS import	Bootstrap styling visible
Image broken	Wrong public path / missing file	Open image path directly in browser	Image loads
Hook rule warning/error	Hook called conditionally or in handler	Inspect Hook placement	Hooks at top level

Step 23 - Configure an npm Run Configuration in IntelliJ

1. Open Run -> Edit Configurations.
2. Click + and choose npm (name may vary with IntelliJ edition/plugins).
3. Set package.json to the project package.json.
4. Set Command/Script to run dev (or the equivalent npm script UI).

5. Name it SBA301 - React Dev.
6. Apply and Run.
7. Confirm the result is equivalent to npm run dev in the terminal.

Checkpoint	Expected result	Student evidence
IntelliJ Run Configuration	The configured run target starts the Vite development server successfully.	PASS / FAIL + note

Step 24 - Production build and preview check

```
npm run build
```

Checkpoint	Expected result	Student evidence
Build	Build finishes without blocking compile error; dist output is produced by the configured Vite build.	PASS / FAIL + note

```
npm run preview
```

Open the preview URL printed by the command and repeat a small smoke test: initial render, search, detail modal, favorite toggle, Context text.

Checkpoint	Expected result	Student evidence
Preview	Built application can be previewed and core interactions still work.	PASS / FAIL + note

Step 25 - Git checkpoint

```
git status
git add .
git commit -m "Build interactive orchid explorer with props state and context"
git log -1
```

Checkpoint	Expected result	Student evidence
Git evidence	Latest commit message and hash are recorded. node_modules is not staged when standard .gitignore is present.	PASS / FAIL + note

Step 26 - Create the evidence package

Evidence	Required content
E01 Environment	node -v, npm -v, IntelliJ project path
E02 Baseline	Initial npm run dev PASS
E03 Component Tree	Final App -> children hierarchy
E04 Props Trace	One orchid value from data source to Card UI
E05 State Trace	One button event to updated UI
E06 Context Trace	Provider -> Navbar/Footer consumers
E07 Browser	Initial UI + search/filter + Modal screenshot(s)
E08 Console	No relevant error after final test
E09 Build	npm run build PASS
E10 Git	git log -1 hash/message

Step 27 - Optional useEffect preview challenge

Only after the core project passes, strong students may connect one state-derived value to the external browser document title. This is a preview, not the main Slot 04 outcome.

```
import { useEffect } from "react";

useEffect(() => {
  document.title = `Orchids (${visibleOrchids.length})`;
}, [visibleOrchids.length]);
```

Required explanation: The external system here is document.title in the browser. Do not use useEffect just because a component renders.

9. Data-flow trace worksheets

9.1 Props trace

Question	Student answer
Value selected for trace	_____
Defined in	_____
Parent component	_____
Prop name	_____
Child component	_____
Rendered at	_____
Why child must not modify it	_____

9.2 State trace

Question	Student answer
State variable	_____
Owner component	_____
Initial value	_____
User event	_____
Handler/updater	_____
New value	_____
Observed UI change	_____

9.3 Context trace

Question	Student answer
Context object	_____
Provider component	_____
Provided value	_____
Consumer 1	_____
Consumer 2	_____
Hook used	_____
Could Props still work? Why/why not?	_____

10. Props vs State vs Context decision guide

Question	Likely choice	Reason
Does the parent already own the value and a direct child needs it?	Props	Explicit parent-to-child input
Does one component need to remember a changing interaction?	Local State	Keep ownership close to where it is used
Can the value be calculated from existing data/state?	Derived data	Avoid unnecessary duplicate State
Do distant descendants genuinely need the same shared value?	Consider Context	Avoid excessive prop drilling when justified
Does the value need database persistence?	Not useState/Context alone	Persistence belongs to later backend/data flow

11. Rules of Hooks verification

Before submission, inspect every Hook call in the project.

Rule check	PASS / FAIL
useState/useContext/useEffect (if used) are called at top level of React function components	

Rule check	PASS / FAIL
No Hook is called inside if/for/while	
No Hook is created inside an event handler	
No Hook is called from a normal non-React utility function	
Hook imports are correct	

Examples that must not appear

```
// INVALID
if (condition) {
  const [show, setShow] = useState(false);
}

// INVALID
function handleClick() {
  const [count, setCount] = useState(0);
}
```

12. Manual test plan

ID	Scenario	Action	Expected result
TC01	Start dev server	npm run dev	Server starts; URL available
TC02	Initial screen	Open URL	All six records visible
TC03	Search name	Type "White"	White Moon remains
TC04	Search category	Type "Dendrobium"	Dendrobium records remain
TC05	Case-insensitive search	Type "phalaenopsis"	Matching cards shown
TC06	No results	Type impossible text	Info Alert appears
TC07	Special filter	Turn switch ON	Only special orchids shown
TC08	Combined filter	Search + special ON	Both conditions applied
TC09	Open modal	Click first card View Detail	Correct details visible
TC10	Close modal	Click Close / X	Modal closes
TC11	Favorite on	Click Add Favorite	Card changes to Favorite
TC12	Favorite off	Click Favorite again	Returns to Add Favorite
TC13	Local-state isolation	Favorite one card	Other cards unchanged
TC14	Context Navbar	Observe greeting	currentUser displayed
TC15	Context Footer	Observe footer	same currentUser displayed
TC16	Refresh behavior	Refresh browser	Local state may reset; source list reloads
TC17	Responsive	Resize browser	Grid stays usable
TC18	Console	Inspect Console	No relevant error
TC19	Build	npm run build	PASS
TC20	Preview	npm run preview	Core smoke tests PASS

13. Common mistakes and corrections

Mistake	Why it is wrong	Correction
Modifying orchid.orchidName inside OrchidCard	Props are read-only for the receiving child	Change ownership/source; do not mutate received Prop
Creating six OrchidCard component files	Destroys reusable-component goal	Use one component + different object Props
Putting visibleOrchids in State unnecessarily	Duplicates information already derivable	Compute it from source data + filter State
Putting every value in Context	Hides simple local/direct data flow	Prefer Props/local State unless sharing is justified
Assuming Favorite persists after refresh	useState is not database persistence	Treat as client state; persistence comes later
Calling setShowDetail(true) directly in JSX render	Can cause immediate repeated updates	Wrap it in an event handler
Calling useState in if	Violates Hook call-order rule	Call Hook at top level
Using AI to rewrite entire project after one error	Loses evidence of the failing stage	Trace source->prop->state->event->render first

14. Human Verification Gate - no AI

1. What is the owner of the orchids array in this project?
2. Why is orchid passed to OrchidCard as a Prop?
3. Why must OrchidCard not modify the received orchid Prop?
4. Why does one OrchidCard implementation display many different orchids?
5. Who owns showDetail and why?
6. Trace the exact steps after clicking View Detail.
7. Who owns isFavorite in the chosen design?
8. Why does favoriting one Card not have to affect another Card?
9. Who owns searchText and specialOnly?
10. Why is visibleOrchids derived data rather than required State?
11. What is the difference between Props and State in this project?
12. What is the difference between State and Context in this project?
13. What value is shared through UserContext?
14. Where is the Provider?
15. Which components consume the Context?
16. Could the current-user value be passed by Props instead? Explain.
17. Why is Context not a database?
18. What two simplified Rules of Hooks are enforced here?
19. Why is useState invalid inside a condition?
20. Which evidence do you inspect first if the UI does not update?
21. Why does React update the UI after the state updater runs?
22. What happens to local Favorite state after a full browser refresh in this project?
23. What does npm run build verify that visual inspection alone does not?
24. Why is a Git commit created after a meaningful working milestone?
25. What would need to change later to persist favorites in SQL Server?

Completion rule: If the application runs but the student cannot explain Props ownership, State update trace, and Context scope, Slot 04 mastery is not demonstrated.

15. AI-Augmented learning workflow

Use AI only after an AI-0 reasoning attempt and evidence collection. AI should review ownership/data flow, not replace the project.

15.1 AI-0 worksheet

Value	My classification	Owner	Reason
orchid object	Prop / State / Context / Derived	_____	_____
showDetail	Prop / State / Context / Derived	_____	_____
isFavorite	Prop / State / Context / Derived	_____	_____
searchText	Prop / State / Context / Derived	_____	_____
visibleOrchids	Prop / State / Context / Derived	_____	_____
currentUser	Prop / State / Context / Derived	_____	_____

15.2 AI ownership review prompt

Act as a React tutor for SBA301 Slot 04.

Here is my component tree:

[PASTE TREE]

Here are the values I use:
[PASTE VALUES]

My classification:
Props: [...]
Local State: [...]
Context: [...]
Derived data: [...]

Do not generate code first.
For each value, check:
1. where it originates;
2. who owns it;
3. who needs it;
4. who may change it;
5. whether my chosen mechanism is the simplest justified option.

Keep the solution within Props, useState, Context/useContext.
Do not introduce Redux, Zustand, Router, backend, or database.

15.3 AI state-debug prompt

My React UI does not update.

Expected: [EXPECTED]
Actual: [ACTUAL]
State owner: [COMPONENT]
Initial state: [VALUE]
User event: [EVENT]
Handler/updater: [CODE]
Rendered JSX: [CODE]
Browser Console: [ERROR OR NONE]

Do not rewrite the component.
Trace only:
Event -> Handler -> State Updater -> New State -> Rendered JSX.
Identify the FIRST broken stage and give one diagnostic action.

15.4 AI Context review prompt

I am using React Context for this value:
[VALUE]

Component tree:
[TREE]

Provider:
[LOCATION]

Consumers:
[LIST]

Do not assume Context is necessary.
Evaluate:
- could direct Props be simpler?
- is real prop drilling occurring?
- is this value genuinely shared?
- is ownership still clear?
Recommend the simplest design.

16. Project completion checklist

- ☐ Project opens correctly in IntelliJ IDEA.
- ☐ node -v and npm -v execute.
- ☐ npm run dev passes.
- ☐ React-Bootstrap is available and Bootstrap CSS is imported.
- ☐ Orchid source data is in one data module.
- ☐ OrchidCard receives an orchid object through Props.
- ☐ Props are not mutated in the child.
- ☐ One OrchidCard implementation renders all records.
- ☐ View Detail uses local useState and works.
- ☐ Favorite uses local useState and works independently.
- ☐ Search input changes State and filtered results update.
- ☐ Special-only switch changes State and filtered results update.
- ☐ visibleOrchids is explained as derived data.
- ☐ Current user is provided through Context.
- ☐ Navbar and Footer consume the same Context.
- ☐ No Hook is called conditionally or inside an event handler.
- ☐ Browser Console has no relevant error.
- ☐ All manual test cases are executed or documented.
- ☐ npm run build passes.
- ☐ npm run preview smoke test passes.
- ☐ Git commit exists and hash is recorded.
- ☐ Props, State, Context trace worksheets are completed.
- ☐ Human Verification Gate can be answered without AI.

17. Quick command sheet

```
# Check environment
node -v
npm -v
git --version

# Install dependencies (only if needed)
npm install
npm install react-bootstrap bootstrap

# Development
npm run dev

# Build verification
npm run build
npm run preview

# Git checkpoint
git status
git add .
git commit -m "Build interactive orchid explorer with props state and context"
git log -1
```

18. Bridge to the next learning unit

After Slot 04, students should no longer view React as static JSX. They should be able to explain a working data-flow chain: source data -> Props -> child UI, and user event -> State updater -> re-render -> updated UI. Context should be recognized as a selective shared-access mechanism, not the default replacement for Props.

```

SLOT 03
I can structure UI as components.
    |
    v
SLOT 04
I can make component data flow and change.
    |
    v
NEXT INTEGRATED LAB
I can combine component architecture + React-Bootstrap + Props + State + Hooks
into one complete React application.
  
```

Appendix A - Complete reference source map

File	Responsibility	Key Slot 04 concept
src/data/orchids.js	Single practice data source	source data
src/context/UserContext.js	Defines Context object	createContext
src/App.jsx	Composition + Provider	Context Provider / architecture
src/components/AppNavbar.jsx	Header + user display	useContext
src/components/HeroSection.jsx	Static intro UI	single responsibility
src/components/OrchidExplorer.jsx	Collection + search/filter	useState + derived data + Props
src/components/OrchidCard.jsx	One reusable orchid item	object Props + local State + events
src/components/AppFooter.jsx	Footer + user display	useContext
src/main.jsx	React root + CSS imports	entry point
src/index.css	Minimal local styling	presentation

Appendix B - Suggested self-assessment rubric

Area	0 - Not demonstrated	1 - Partial	2 - Demonstrated
Props	Copied/hard-coded cards	Prop exists but flow unclear	Reusable object Prop; flow explained
State	No interaction / direct mutation	Interaction works but ownership unclear	Owner + updater + render trace explained
Context	Misused as global default	Works but justification weak	Provider/consumer clear; use justified/proportional
Hooks	Rule violations	No violation but cannot explain	Top-level rules explained and followed
Debugging	Guess/recreate project	Uses some error evidence	Traces first broken stage systematically
Verification	Visual only	Some tests	Manual tests + Console + build + preview evidence
Git	No checkpoint	Commit exists	Meaningful commit + hash recorded

Final student statement: I can explain where data comes from, who owns it, who may change it, how user events update State, and why Context is or is not justified in this application.